Task 6

Deploy the Trained Model (Flask REST API)

Predictive Analytics & AI Model Deployment - Customer Churn Prediction

Objective. Serve the trained model through a REST API and web interface, ready for Docker / cloud deployment.

1. Deployment design

- **Flask** application factory (`app.app:create_app`) loads the saved best pipeline **once** at start-up and fails fast if artifacts are missing.
- The saved pipeline contains all preprocessing, so the API accepts **raw customer fields** (missing values, messy text and out-of-range numbers are handled exactly as in training).
- Input validation: three required fields (tenure_months, monthly_charges, contract_type), numeric coercion, 5 MB body limit, 5,000-row batch limit; errors return JSON with HTTP 400/413.
- Response: churn probability, decision (threshold 0.30), and risk level Low/Medium/High.

2. Endpoints

Method	Endpoint	Description
GET	/	Web form: enter customer details, see churn probability and risk level
GET	/health	Liveness probe: {status: ok, model: ...}
GET	/model-info	Model name, threshold, test metrics, required/optional fields
POST	/predict	One customer (JSON) -> probability, label, risk level
POST	/predict_batch	JSON list or multipart CSV upload (field 'file'), max 5,000 rows
GET	/dashboard	Analytics dashboard (Task 7)

3. Live example (real HTTP call against the running server)

Request

```
curl -X POST http://localhost:5000/predict -H 'Content-Type: application/json' \
  -d '{"tenure_months":3,"monthly_charges":95,"total_charges":285,
      "contract_type":"Monthly","internet_service":"Fiber",
      "payment_method":"Electronic Check","satisfaction_score":2,
      "num_support_tickets":5,"late_payments":3}'
```

Response

```
{"churn_probability":0.9787,"label":"Will churn","prediction":1,"risk_level":"High","threshold":0.3}
```

Validation error (missing fields)

```
{"error":"missing required field(s): monthly_charges, contract_type"}
```

4. Automated tests

Seven tests (health, pages, high- vs low-risk ordering, messy input, validation errors, JSON batch, CSV batch) using Flask's test client. Result:

```
test_batch (tests.test_api.ApiTests.test_batch) ... ok
test_batch_csv_upload (tests.test_api.ApiTests.test_batch_csv_upload) ... ok
test_health (tests.test_api.ApiTests.test_health) ... ok
ok
test_messy_input_is_handled (tests.test_api.ApiTests.test_messy_input_is_handled) ... ok
```

```
test_predict_high_vs_low (tests.test_api.ApiTests.test_predict_high_vs_low) ... ok
test_validation_errors (tests.test_api.ApiTests.test_validation_errors) ... ok

----------------------------------------------------------------------
Ran 7 tests in 0.601s

OK
```

5. How to run and deploy

Target	Command / file
Local development	python -m app.app -> http://127.0.0.1:5000
Production (WSGI)	gunicorn -b 0.0.0.0:8000 -w 2 "app.app:create_app()"
Docker	Dockerfile: docker build -t churn-api . && docker run -p 8000:8000 churn-api
Cloud PaaS (Render, Heroku, Railway, ...)	Procfile + requirements.txt are included; connect the GitHub repo and deploy
Cloud containers (AWS ECS/App Runner, GCP Cloud Run, Azure)	Push the Docker image and expose port 8000

Note: the API was tested locally and with the test client here; deployment to a public cloud requires the reader's own cloud account. Add authentication, HTTPS and monitoring before exposing it publicly.

Appendix - Source code

app/app.py

```
"""TASK 6 - Model deployment: Flask REST API + web form + dashboard host.

Run locally:      python -m app.app                (http://127.0.0.1:5000)
Run production:   gunicorn -b 0.0.0.0:8000 "app.app:create_app()"
"""
import io
import pandas as pd
from flask import Flask, jsonify, render_template, request, send_from_directory
from src.config import REPORTS
from src.predict import load_artifacts, predict_records

REQUIRED = ["tenure_months", "monthly_charges", "contract_type"]
NUMERIC_FIELDS = ["age", "tenure_months", "monthly_charges", "total_charges", "num_products",
                  "num_support_tickets", "late_payments", "avg_monthly_usage_gb",
                  "satisfaction_score", "last_login_days", "discount_offer"]
MAX_BATCH = 5000


def validate(rec, idx=None):
    where = f"record {idx}: " if idx is not None else ""
    if not isinstance(rec, dict):
        return f"{where}each record must be a JSON object"
    missing = [f for f in REQUIRED if rec.get(f) in (None, "")]
    if missing:
        return f"{where}missing required field(s): {', '.join(missing)}"
    for f in NUMERIC_FIELDS:
        if rec.get(f) not in (None, ""):
            try:
                float(rec[f])
            except (TypeError, ValueError):
                return f"{where}field '{f}' must be numeric"
    return None


def create_app():
    app = Flask(__name__)
    app.config["MAX_CONTENT_LENGTH"] = 5 * 1024 * 1024
    model, meta = load_artifacts()          # fail fast if artifacts are missing

    @app.get("/")
    def index():
        return render_template("index.html", meta=meta)

    @app.get("/health")
    def health():
        return jsonify(status="ok", model=meta["best_model_name"])

    @app.get("/model-info")
    def model_info():
        return jsonify(model=meta["best_model_name"], threshold=meta["threshold"],
                       test_metrics=meta["test_metrics_at_threshold"], trained_at=meta["trained_at"],
                       required_fields=REQUIRED, numeric_fields=NUMERIC_FIELDS,
                       categorical_fields=["gender", "region", "contract_type", "internet_service", "payment_method"])

    @app.post("/predict")
    def predict():
        rec = request.get_json(silent=True)
        err = validate(rec)
        if err:
            return jsonify(error=err), 400
        return jsonify(predict_records([rec])[0])

    @app.post("/predict_batch")
    def predict_batch():
        if request.files.get("file"):                       # CSV upload
            try:
                df = pd.read_csv(io.BytesIO(request.files["file"].read()))
            except Exception:
                return jsonify(error="could not parse CSV"), 400
            records = df.where(df.notna(), None).to_dict("records")
        else:
            records = request.get_json(silent=True)
        if not isinstance(records, list) or not records:
            return jsonify(error="send a non-empty JSON list of records or a CSV file"), 400
        if len(records) > MAX_BATCH:
            return jsonify(error=f"batch too large (max {MAX_BATCH})"), 413
        for i, r in enumerate(records):
            err = validate(r, i)
            if err:
                return jsonify(error=err), 400
        results = predict_records(records)
        return jsonify(count=len(results), results=results)

    @app.get("/dashboard")
    def dashboard():
        return send_from_directory(REPORTS, "dashboard.html")

    return app
```

```
if __name__ == "__main__":
    create_app().run(host="0.0.0.0", port=5000, debug=False)
```

src/predict.py

```
"""Inference helpers shared by the Flask API, tests and notebooks."""
import json
from functools import lru_cache
import joblib
import pandas as pd
from .config import MODELS, RISK_BINS, RISK_LABELS


@lru_cache(maxsize=1)
def load_artifacts():
    model = joblib.load(MODELS / "best_model.joblib")
    meta = json.loads((MODELS / "metadata.json").read_text())
    return model, meta


def risk_level(p: float) -> str:
    for lo, hi, lab in zip(RISK_BINS[:-1], RISK_BINS[1:], RISK_LABELS):
        if lo <= p < hi:
            return lab
    return RISK_LABELS[-1]


def predict_records(records):
    """records: list[dict] with raw customer fields -> list of prediction dicts."""
    model, meta = load_artifacts()
    df = pd.DataFrame(records)
    probs = model.predict_proba(df)[:, 1]
    return [{"churn_probability": round(float(p), 4), "prediction": int(p >= meta["threshold"]),
             "label": "Will churn" if p >= meta["threshold"] else "Will stay",
             "risk_level": risk_level(float(p)), "threshold": meta["threshold"]} for p in probs]
```

tests/test_api.py

```
"""API tests (Flask test client). Run: python -m unittest tests.test_api -v"""
import unittest
from app.app import create_app

HIGH_RISK = {"tenure_months": 3, "monthly_charges": 95, "total_charges": 285, "contract_type": "Monthly",
             "internet_service": "Fiber", "payment_method": "Electronic Check", "satisfaction_score": 2,
             "num_support_tickets": 5, "late_payments": 3, "last_login_days": 60, "num_products": 1, "age": 28}
LOW_RISK = {"tenure_months": 60, "monthly_charges": 55, "total_charges": 3300, "contract_type": "Two-Year",
            "internet_service": "DSL", "payment_method": "Credit Card", "satisfaction_score": 9,
            "num_support_tickets": 0, "late_payments": 0, "last_login_days": 2, "num_products": 4,
            "discount_offer": 1, "age": 52}


class ApiTests(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        cls.c = create_app().test_client()

    def test_health(self):
        r = self.c.get("/health"); self.assertEqual(r.status_code, 200); self.assertEqual(r.json["status"], "ok")

    def test_home_and_dashboard(self):
        self.assertEqual(self.c.get("/").status_code, 200)
        self.assertEqual(self.c.get("/dashboard").status_code, 200)

    def test_predict_high_vs_low(self):
        hi = self.c.post("/predict", json=HIGH_RISK).json; lo = self.c.post("/predict", json=LOW_RISK).json
        self.assertGreater(hi["churn_probability"], lo["churn_probability"])
        self.assertEqual(hi["risk_level"], "High"); self.assertEqual(lo["risk_level"], "Low")

    def test_messy_input_is_handled(self):
        r = self.c.post("/predict", json={"tenure_months": "12", "monthly_charges": "70", "contract_type": " monthly "})
        self.assertEqual(r.status_code, 200); self.assertTrue(0 <= r.json["churn_probability"] <= 1)

    def test_validation_errors(self):
        self.assertEqual(self.c.post("/predict", json={"tenure_months": 5}).status_code, 400)
        bad = dict(LOW_RISK, monthly_charges="abc")
        self.assertEqual(self.c.post("/predict", json=bad).status_code, 400)
        self.assertEqual(self.c.post("/predict", data="not json").status_code, 400)

    def test_batch(self):
        r = self.c.post("/predict_batch", json=[HIGH_RISK, LOW_RISK])
        self.assertEqual(r.status_code, 200); self.assertEqual(r.json["count"], 2)
        self.assertEqual(self.c.post("/predict_batch", json=[]).status_code, 400)

    def test_batch_csv_upload(self):
        import io
        csv = "tenure_months,monthly_charges,contract_type\n5,90,Monthly\n50,40,Two-Year\n"
        r = self.c.post("/predict_batch", data={"file": (io.BytesIO(csv.encode()), "c.csv")},
                        content_type="multipart/form-data")
        self.assertEqual(r.status_code, 200); self.assertEqual(r.json["count"], 2)


if __name__ == "__main__":
```

```
    unittest.main()
```

Dockerfile

```
FROM python:3.12-slim
WORKDIR /srv
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["gunicorn", "-b", "0.0.0.0:8000", "-w", "2", "app.app:create_app()"]
```